Dimensional Data Modeling Homework Submission

Grade: A

Status: Pass (automatic by LLM)

Feedback

Thanks for the clear, well-organized submission. Overall, you've implemented a clean dimensional model and both full-backfill and incremental SCD Type 2 logic correctly. The intent of each query is well documented, your use of composite/enum types is thoughtful, and your incremental SCD handling (extend unchanged intervals, split on change, start intervals for new actors) is solid.

What you did well
- Clear grain definitions and comments across all files.
- Cumulative actors table correctly models "one row per actor per year" and appends films while preserving history.
- Sensible quality_class recalculation policy: only when there are new films; otherwise retain prior classification.
- FULL OUTER JOIN in query_2 ensures you include carried-forward actors and brand-new ones.
- Type 2 SCD table DDL is fit-for-purpose, with a simple integer time axis and a CHECK on date bounds.
- Full backfill (query_4) leverages LAG and a streak-identifier pattern correctly to collapse contiguous states.
- Incremental SCD (query_5) is well-structured: you copy historicals, extend unchanged current intervals, split on change, and add new actor intervals. The use of a composite type to UNNEST paired rows for changed records is both elegant and efficient.

Suggestions and potential improvements
Schema and data types
- Consider making fields NOT NULL where business rules allow:
- actors.actor (likely NOT NULL)
- actors.is_active (likely NOT NULL)
- actors.quality_class (if you never want unknown classification)
- actors.films with a default empty array: DEFAULT '{}'::film_struct[]
- rating as REAL can introduce precision quirks. Prefer DOUBLE PRECISION or NUMERIC(3,1) for ratings and ensure AVG(rating) matches that precision.
- If you anticipate name changes, you may eventually want a separate, static actors_dim keyed by actorid and track name changes there (or add a separate SCD for name, if needed).

Query 2 (cumulative actors)
- Year parameters: Hardcoding 2020/2021 is fine for an example, but consider parameterizing years (e.g., a small params CTE or runtime variables) to make the scripts reusable.
- Films array ordering and dedup:
- You ORDER BY filmid inside ARRAY_AGG for the incoming year but not for the cumulative result after concatenation. If consistent ordering matters, rebuild the array from UNNEST(ly.films || ty.films) with ORDER BY filmid and optionally DISTINCT on filmid to avoid accidental duplicates.

- Thresholds: You're using > for cutoffs. If business rules require inclusivity (e.g., 8.0 should be star), change to >= as appropriate.
- Re-runs: INSERT without ON CONFLICT will fail on a second run for the same year due to the PK. That may be desirable. If you want idempotence, consider TRUNCATE for that year or add ON CONFLICT DO NOTHING/UPDATE as per your policy.

Query 3 (SCD DDL)
- Your PK includes current_year, which cleanly separates SCD snapshots across runs. If you want to prevent interval overlaps within a snapshot, consider adding an additional constraint (e.g., an exclusion or a unique constraint on ranges) in more advanced schemas. For this assignment, your approach is fine.
- Consider indexing for common lookups:
- actors_history_scd(actorid, current_year)
- actors_history_scd(actorid, start_date, end_date)

Query 4 (full backfill)
- The did_change logic works. To make the first row per actor explicitly start a new streak, you could coalesce it with "first row" logic (e.g., row_number()=1 OR did_change). Your current use of IS DISTINCT FROM should already flag the first row as a change in typical data, but being explicit can make intent clearer.
- latest_year cross join is a nice touch to stamp the snapshot.

Query 5 (incremental SCD)
- Logic is correct and clean:
- historical_scd: retains closed intervals from prior snapshot.
- unchanged_records: extends open intervals.
- changed_records: splits intervals.
- new_records: starts new intervals for brand-new or reappearing actors.
- This depends on last_year_scd being "open as of 2020" (end_date=2020). Good pattern. As with query_2, consider parameterizing prev/curr years.
- Re-run behavior: As with query_2, re-inserting the same snapshot will duplicate unless constrained. You rely on the PK (actorid,start_date,current_year) to prevent some duplication, but you can still encounter conflicts. Decide whether you want to:
- TRUNCATE actors_history_scd for the snapshot year before re-run, or
- Use INSERT ... ON CONFLICT to upsert for the snapshot year.

Edge cases to think about
- Missing years: If an actor has gaps in the actors table, your full backfill will treat the gap as continuous. This is fine if your cumulative snapshot generates rows every year; otherwise, clarify policy (e.g., treat gaps as continuous state or break streaks).
- Null ratings in the incoming year: AVG ignores nulls, which is likely what you want. If all ratings are null, AVG returns null; your CASE treats this as ELSE -> 'bad'. If you prefer to retain prior quality_class in such a case, wrap AVG in COALESCE.
- Performance/scalability: Add appropriate indexes on source (actor_films(year, actorid)) and on actors(actorid, current_year) to keep joins and groupings snappy.

What I'd need from you if something is off in my review
- The exact expected business rules for quality_class thresholds (strict vs inclusive).
- A brief DDL for actor_films (column names/types) and a small sample dataset to validate array ordering/dedup behavior.

- Clarification on whether you expect idempotent re-runs and how you prefer to handle them (truncate, upsert, or fail).
- Confirmation that actors exists for every year (i.e., cumulative snapshots are annual and contiguous) or whether gaps can occur.
- Whether you want films arrays to be unique by filmid and consistently ordered across years.

Overall assessment
This is a strong submission that shows clear understanding of cumulative dimensional modeling and SCD Type 2 handling. My suggestions are mostly about robustness (parameterization, idempotence, array ordering/dedup, and a few constraints/indexes). Nice work.